PYTHON PROGRAMMING PROJECT REPORT

Ten Programs with Problem Statements, Objectives, Assumptions, Algorithms, Flowcharts, Pseudocode, Source Code, Test Cases, Outputs and Conclusions

Submitted by CHAVADAM SRI SANJANA

Roll Number 26001199

Department / Class CSM

Academic Year 2026

1. Retail Billing with Multiple Discount Rules

1. Problem Statement

Develop a Python program to calculate a customer's retail bill for multiple products. The program must accept each product's price and quantity, calculate the subtotal, apply a 10% discount when the subtotal is at least ■2,000 (with a maximum discount of ■500), add 5% tax, and display the final bill.

2. Objectives

• Accept the number of products and each product's price and quantity.

• Calculate the total cost of all products.

• Apply the discount condition and maximum discount limit.

• Calculate tax and display the final bill amount.

3. Assumptions

• The number of products entered is a positive integer.

• Product prices are non-negative and quantities are positive integers.

• The discount is calculated on the subtotal before tax.

• A 5% tax is applied to the original subtotal.

• The discount is 10% only when the subtotal is ■2,000 or more, and it cannot exceed ■500.

4. Algorithm

1. Start the program.

2. Read the number of products and initialize total to zero.

3. Repeat for each product: read price and quantity, calculate price × quantity, and add it to total.

4. If total is at least 2,000, calculate a 10% discount; otherwise set discount to zero.

5. Limit the discount to a maximum of 500.

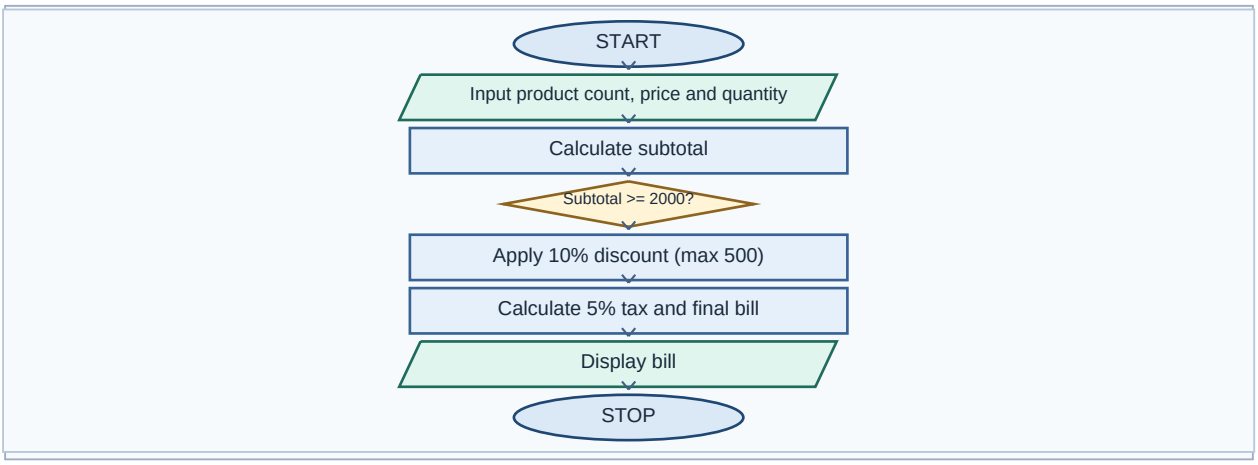
6. Calculate tax as 5% of total.

7. Calculate the final bill as total – discount + tax.

8. Display total, discount, tax, and final bill.

9. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
total ← 0
FOR each product from 1 to n
    INPUT price, quantity
    total ← total + (price × quantity)
END FOR
IF total >= 2000
    discount ← total × 0.10
ELSE
    discount ← 0
END IF
discount ← MIN(discount, 500)
tax ← total × 0.05
bill ← total - discount + tax
OUTPUT total, discount, tax, bill
STOP
```

7. Python Source Code

```
# Retail Billing with Multiple Discount Rules
n = int(input("Enter number of products: "))
total = 0

for i in range(n):
    price = float(input("Enter price: "))
    qty = int(input("Enter quantity: "))
    total += price * qty

discount = total * 0.10 if total >= 2000 else 0
discount = min(discount, 500)
tax = total * 0.05
bill = total - discount + tax

print("Total:", total)
print("Discount:", discount)
print("Tax:", tax)
print("Final Bill:", bill)
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	■1,000 × 1 and ■500 × 1	■1,500 | ■0 | ■75 | ■1,575
2	■2,000 × 1	■2,000 | ■200 | ■100 | ■1,900
3	■6,000 × 1	■6,000 | ■500 (capped) | ■300 | ■5,800

9. Output

```
Enter number of products: 2
Enter price: 1200
Enter quantity: 1
Enter price: 1000
Enter quantity: 1
Total: 2200.0
Discount: 220.0
Tax: 110.0
Final Bill: 2090.0
```

10. Conclusion

The program successfully calculates the retail bill for multiple products and applies the specified discount and tax rules. It demonstrates loops, arithmetic operations, conditional expressions, and the min() function in Python.

2. Retail Billing with Multiple Discount Rules (Repeated Exercise)

1. Problem Statement

Write a Python program that accepts multiple product prices and quantities, finds the subtotal, applies a conditional 10% discount for bills of ■2,000 or above subject to a ■500 cap, adds 5% tax, and prints the bill details.

2. Objectives

• Read product details from the user.

• Calculate the combined product cost.

• Apply the discount threshold and maximum limit.

• Add tax and display the final payable amount.

3. Assumptions

• The input number of products is a positive integer.

• Prices are non-negative and quantities are positive integers.

• Discount is based on the subtotal and is capped at ■500.

• Tax is 5% of the subtotal and is added after discount calculation.

4. Algorithm

1. Start.

2. Input the number of products and set total to zero.

3. For every product, input price and quantity and add price × quantity to total.

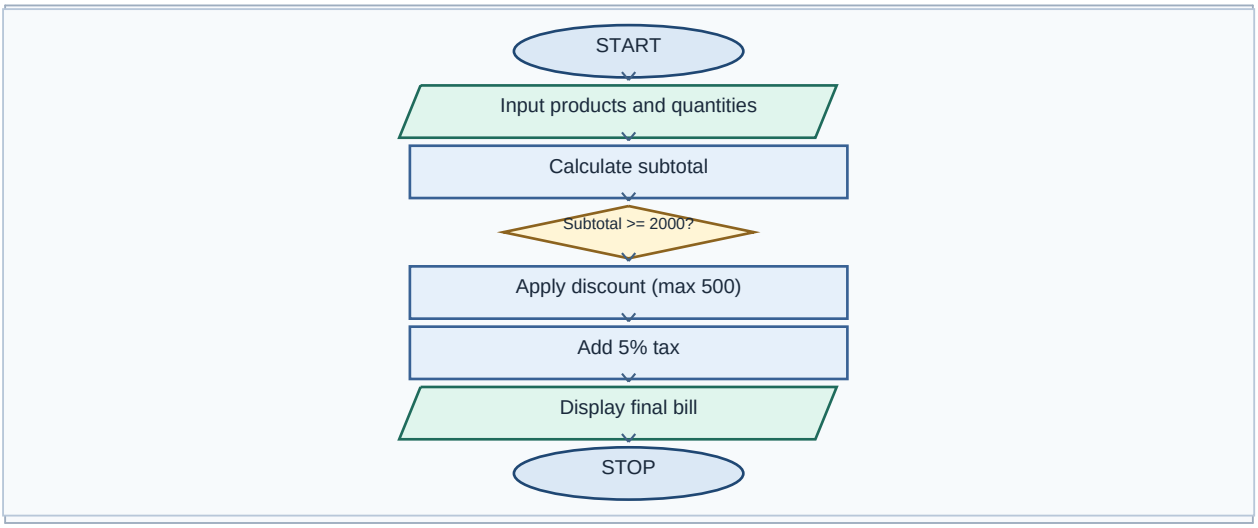
4. Calculate 10% discount if total is at least 2,000; otherwise use zero.

5. Cap the discount at 500.

6. Calculate 5% tax and final bill = total – discount + tax.

7. Print the bill details and stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
total ← 0
```

```
REPEAT n times
    INPUT price, quantity
    total ← total + price × quantity
END REPEAT
discount ← 0
IF total >= 2000
    discount ← total × 0.10
END IF
discount ← MIN(discount, 500)
tax ← total × 0.05
bill ← total - discount + tax
DISPLAY total, discount, tax, bill
STOP
```

7. Python Source Code

```
n = int(input("Enter number of products: "))
total = 0

for i in range(n):
    price = float(input("Enter price: "))
    qty = int(input("Enter quantity: "))
    total += price * qty

discount = total * 0.10 if total >= 2000 else 0
discount = min(discount, 500)
tax = total * 0.05
bill = total - discount + tax

print("Total:", total)
print("Discount:", discount)
print("Tax:", tax)
print("Final Bill:", bill)
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	■1,500 subtotal	■1,500 | ■0 | ■75 | ■1,575
2	■2,500 subtotal	■2,500 | ■250 | ■125 | ■2,375
3	■8,000 subtotal	■8,000 | ■500 (capped) | ■400 | ■7,900

9. Output

```
Enter number of products: 1
Enter price: 2500
Enter quantity: 1
Total: 2500.0
Discount: 250.0
Tax: 125.0
Final Bill: 2375.0
```

10. Conclusion

The program automates retail bill calculation and applies the required discount and tax conditions. It uses iteration, conditional logic, and basic arithmetic to produce the payable amount.

3. Campus Event Financial Planning

1. Problem Statement

Create a Python program to calculate the total expenditure for a campus event using venue, equipment, catering, transport, and certificate costs. After subtracting sponsorship, calculate the funding shortfall and the minimum number of participant registrations required to cover it using the registration fee per participant.

2. Objectives

• Collect all event expense amounts and sponsorship details.

• Calculate the total event cost.

• Find the sponsorship shortfall without producing a negative value.

• Calculate the minimum registrations needed to cover the shortfall.

3. Assumptions

• All costs and sponsorship amounts are non-negative.

• The registration fee per participant is greater than zero.

• Each participant pays the same registration fee.

• A partial participant count is rounded up to the next whole participant.

4. Algorithm

1. Start.

2. Read venue, equipment, catering, transport, and certificate costs.

3. Add all costs to find total expenditure.

4. Read sponsorship and registration fee per participant.

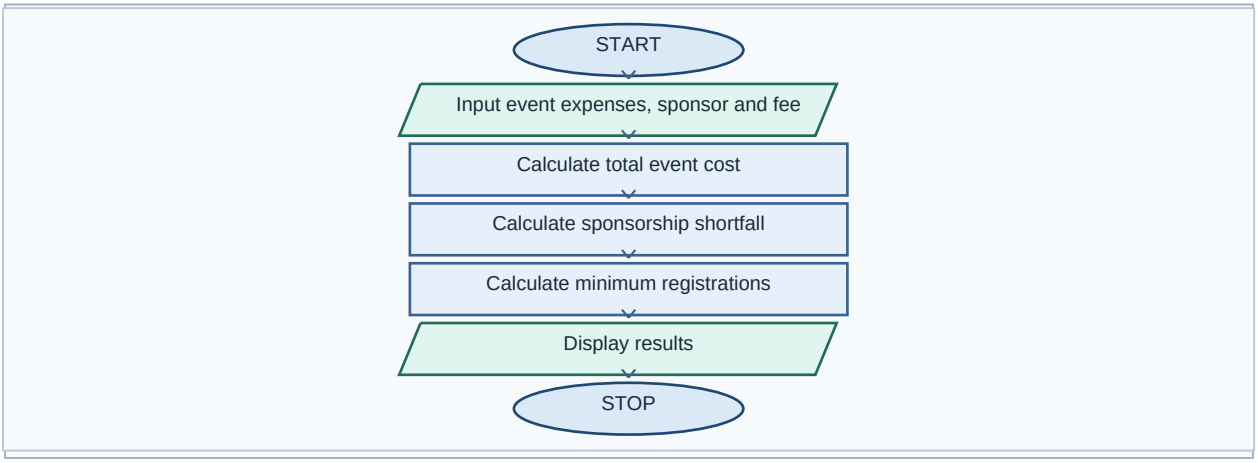
5. Calculate shortfall as the greater of zero and total cost minus sponsorship.

6. Calculate required registrations by dividing shortfall by fee and rounding up.

7. Display total cost, shortfall, participant contribution needed, and registrations.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT venue, equipment, catering, transport, certificates
INPUT sponsor
INPUT fee
```

```
total ← venue + equipment + catering + transport + certificates
shortfall ← MAX(0, total - sponsor)
participants ← CEILING(shortfall / fee)
DISPLAY total, shortfall
DISPLAY shortfall as participant contribution needed
DISPLAY participants
STOP
```

7. Python Source Code

```
# Campus event financial planning
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost: "))
transport = float(input("Enter transport cost: "))
certificates = float(input("Enter certificate cost: "))
sponsor = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per participant: "))

total = venue + equipment + catering + transport + certificates
shortfall = max(0, total - sponsor)
participants = int(shortfall / fee) + (shortfall % fee > 0)

print("Total Cost:", total, "\nSponsorship Shortfall:", shortfall)
print("Participant Contribution Needed:", shortfall)
print("Minimum Registrations to Break Even:", participants)
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	Total ■50,000; sponsor ■20,000; fee ■500	■50,000 | ■30,000 | ■30,000 | 60
2	Total ■30,000; sponsor ■30,000; fee ■500	■30,000 | ■0 | ■0 | 0
3	Total ■10,000; sponsor ■8,800; fee ■500	■10,000 | ■1,200 | ■1,200 | 3

9. Output

```
Enter venue cost: 20000
Enter equipment cost: 5000
Enter catering cost: 15000
Enter transport cost: 3000
Enter certificate cost: 2000
Enter sponsorship amount: 20000
Enter registration fee per participant: 500
Total Cost: 45000.0
Sponsorship Shortfall: 25000.0
Participant Contribution Needed: 25000.0
Minimum Registrations to Break Even: 50
```

10. Conclusion

The program helps organizers estimate event expenses, identify the amount not covered by sponsorship, and determine the minimum registrations needed to break even.

4. Multi-Stage Examination Eligibility System

1. Problem Statement

Develop a Python program to check a student's eligibility for a final examination based on four conditions: minimum attendance of 75%, completion of laboratory work, internal marks of at least 40, and clearance of fees. The program must display every condition that is not satisfied or confirm eligibility when all conditions are met.

2. Objectives

• Accept attendance, lab completion, internal marks, and fee status.

• Check each eligibility condition independently.

• Display all reasons for ineligibility.

• Display an eligibility message only when every condition is satisfied.

3. Assumptions

• Attendance is entered as a percentage from 0 to 100.

• Internal marks are entered as an integer.

• Lab and fee responses are entered as yes or no in lowercase.

• The required attendance is 75% and minimum internal marks are 40.

4. Algorithm

1. Start.

2. Read attendance percentage.

3. Read lab completion status.

4. Read internal marks.

5. Read fee clearance status.

6. If attendance is below 75, display attendance shortage.

7. If lab status is not yes, display lab not completed.

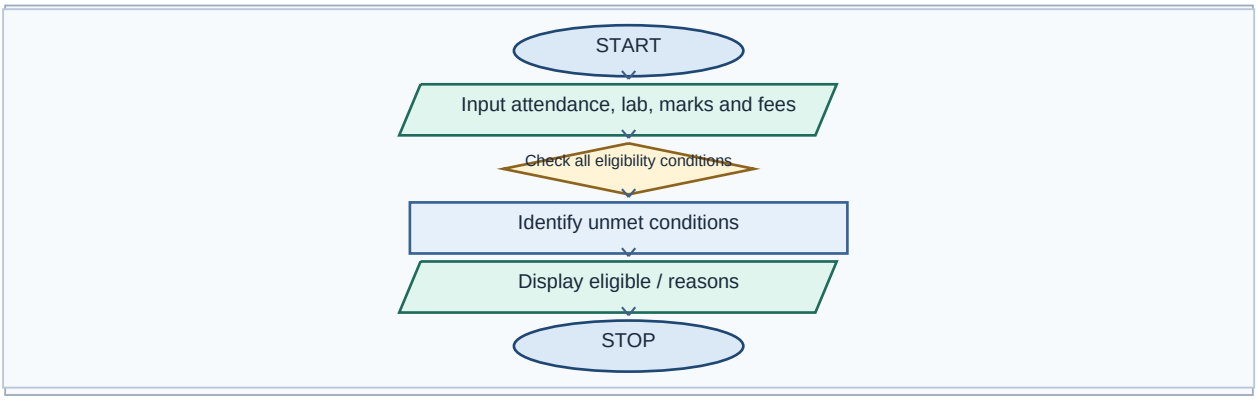
8. If internal marks are below 40, display insufficient marks.

9. If fees are not cleared, display fees not cleared.

10. If all four conditions are satisfied, display eligible for final examination.

11. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT attendance
INPUT lab
INPUT internal
INPUT fees
IF attendance < 75
    DISPLAY "Attendance shortage"
END IF
IF lab != "yes"
    DISPLAY "Lab not completed"
END IF
IF internal < 40
    DISPLAY "Internal marks insufficient"
END IF
IF fees != "yes"
    DISPLAY "Fees not cleared"
END IF
IF attendance >= 75 AND lab = "yes"
   AND internal >= 40 AND fees = "yes"
    DISPLAY "Eligible for final examination"
END IF
STOP
```

7. Python Source Code

```
# Multi-stage examination eligibility system
attendance = int(input("Enter attendance %: "))
lab = input("Lab completed (yes/no): ")
internal = int(input("Enter internal marks: "))
fees = input("Fees cleared (yes/no): ")

if attendance < 75:
    print("Attendance shortage")
if lab != "yes":
    print("Lab not completed")
if internal < 40:
    print("Internal marks insufficient")
if fees != "yes":
    print("Fees not cleared")

if attendance >= 75 and lab == "yes" and internal >= 40 and fees == "yes":
    print("Eligible for final examination")
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	Attendance 80, lab yes, marks 60, fees yes	All conditions satisfied | — | — | Eligible
2	Attendance 70, lab yes, marks 60, fees yes	Attendance below 75% | — | — | Not eligible
3	Attendance 80, lab no, marks 35, fees no	Lab, marks, and fees conditions fail | — | — | Not eligible; all reasons shown

9. Output

```
Enter attendance %: 80
Lab completed (yes/no): yes
Enter internal marks: 65
Fees cleared (yes/no): yes
Eligible for final examination
```

10. Conclusion

The program checks multiple academic requirements independently and reports all unmet conditions. It demonstrates the use of if statements and logical AND operations in decision-making.

5. Bus Passenger Occupancy

1. Problem Statement

Write a Python program to track the number of passengers on a bus across three stops. At each stop, accept the number of passengers boarding and leaving, update the current passenger count, and display the final occupancy after all stops.

2. Objectives

• Initialize the passenger count.

• Read boarding and leaving counts at each stop.

• Update occupancy after every stop.

• Display the final number of passengers.

3. Assumptions

• The bus starts with zero passengers.

• The route has exactly three stops.

• Boarding and leaving values are non-negative integers.

• The number of passengers leaving should not exceed the number currently on the bus; the given code does not validate this condition.

4. Algorithm

1. Start and initialize passengers to zero.

2. Repeat for three stops.

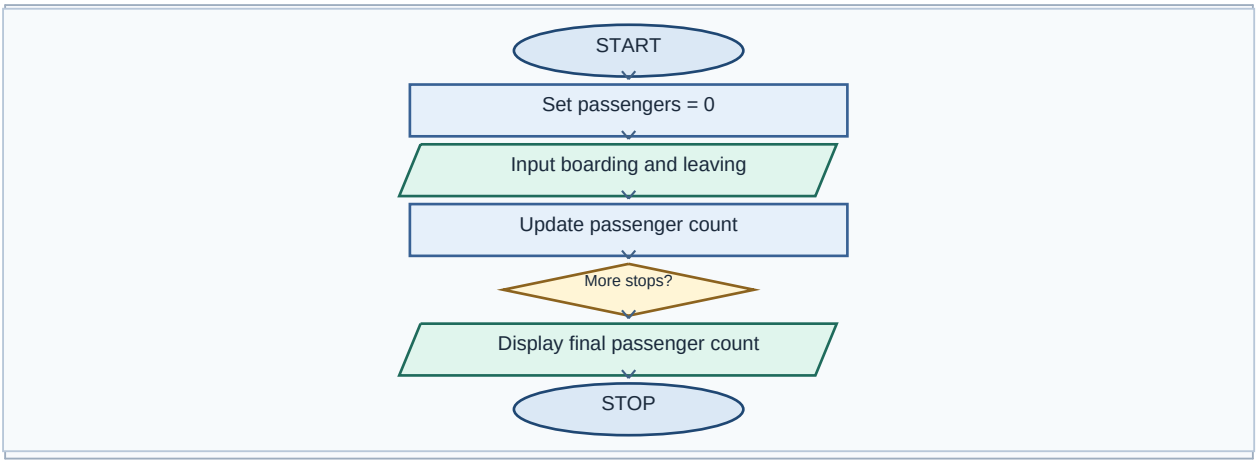
3. Read the number boarding and the number leaving.

4. Update passengers = passengers + boarding – leaving.

5. After all stops, display the passenger count.

6. Stop.

5. Flowchart



6. Pseudocode

```
START
passengers ← 0
FOR stop from 1 to 3
    INPUT board
    INPUT leave
    passengers ← passengers + board - leave
END FOR
DISPLAY passengers
```

```
STOP
```

7. Python Source Code

```
# Bus passenger occupancy
passengers = 0

for stop in range(3):
    board = int(input("Passengers boarding: "))
    leave = int(input("Passengers leaving: "))

    passengers = passengers + board - leave

print("Total passengers:", passengers)
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	Stop 1: +10 – 2; Stop 2: +5 – 3; Stop 3: +4 – 1	10 | — | — | 13
2	No passengers board or leave at all stops	0 | — | — | 0
3	Stop 1: +20 – 0; Stop 2: +0 – 5; Stop 3: +2 – 3	14 | — | — | 14

9. Output

```
Passengers boarding: 10
Passengers leaving: 2
Passengers boarding: 5
Passengers leaving: 3
Passengers boarding: 4
Passengers leaving: 1
Total passengers: 13
```

10. Conclusion

The program tracks bus occupancy by updating the passenger count at each stop. It demonstrates for loops, user input, and arithmetic operations.

6. Energy Conservation Before-and-After Analysis

1. Problem Statement

Create a Python program to compare energy usage before and after a conservation measure. Calculate the number of units saved and the percentage saved. If the initial usage is zero or less, display a message that the percentage cannot be calculated.

2. Objectives

• Accept energy usage before and after conservation.

• Calculate the difference in energy consumption.

• Calculate the percentage saved when initial usage is positive.

• Handle zero or negative initial usage safely.

3. Assumptions

• Energy usage is entered as an integer number of units.

• The before and after values are expected to be non-negative.

• Percentage savings are calculated relative to the before value.

• If before is zero or less, percentage calculation is skipped.

4. Algorithm

1. Start.

2. Read usage before and usage after.

3. Calculate saved units as before minus after.

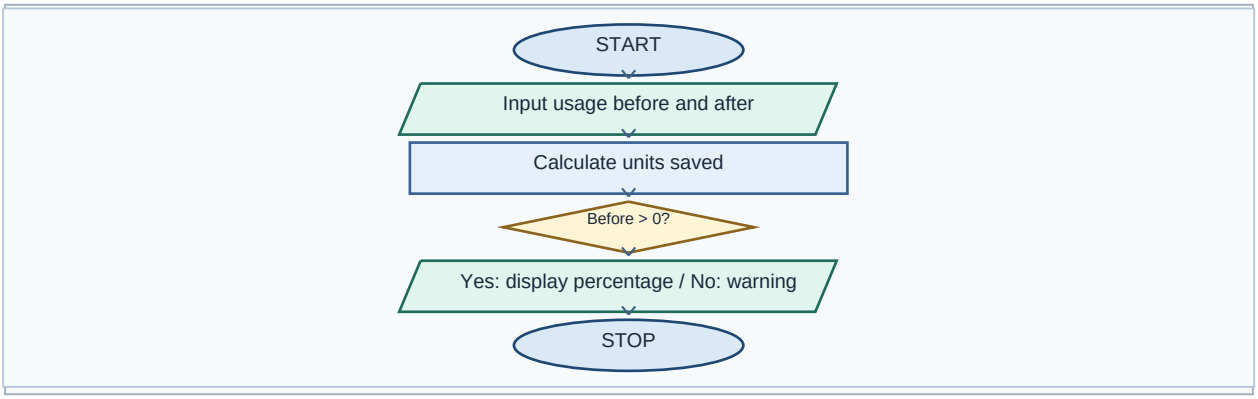
4. Display units saved.

5. If before is greater than zero, calculate saved / before × 100 and display it.

6. Otherwise display cannot calculate percentage.

7. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT before
INPUT after
saved ← before - after
DISPLAY saved
IF before > 0
    percent ← (saved / before) × 100
    DISPLAY percent
ELSE
```

```
    DISPLAY "Cannot calculate percentage"
END IF
STOP
```

7. Python Source Code

```
# Energy conservation before and after analysis
before = int(input("Usage before: "))
after = int(input("Usage after: "))
saved = before - after
print("Units saved:", saved)

if before > 0:
    percent = saved / before * 100
    print("Percentage saved:", percent)
else:
    print("Cannot calculate percentage")
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	Before 500, after 350	150 | 30% | — | 150 units saved
2	Before 100, after 100	0 | 0% | — | No reduction
3	Before 0, after 0	0 | Not calculated | — | Cannot calculate percentage

9. Output

```
Usage before: 500
Usage after: 350
Units saved: 150
Percentage saved: 30.0
```

10. Conclusion

The program compares energy consumption and calculates both the units saved and the percentage reduction. It also avoids division by zero by checking the initial usage.

7. Academic Progression

1. Problem Statement

Develop a Python program to determine whether a student is eligible to progress academically. Eligibility depends on passing the subject with at least 40 marks, having attendance of at least 75%, and completing all prerequisites. The program should display each failed condition or confirm eligibility.

2. Objectives

• Read semester marks and attendance percentage.

• Check whether prerequisites are completed.

• Identify all conditions that are not satisfied.

• Display the progression result.

3. Assumptions

• Marks are entered as an integer from 0 to 100.

• Attendance is entered as a percentage from 0 to 100.

• Prerequisite status is entered as yes or no in lowercase.

• Minimum passing marks are 40 and minimum attendance is 75%.

4. Algorithm

1. Start.

2. Read semester marks.

3. Read attendance percentage.

4. Read prerequisite completion status.

5. If marks are below 40, display subject not passed.

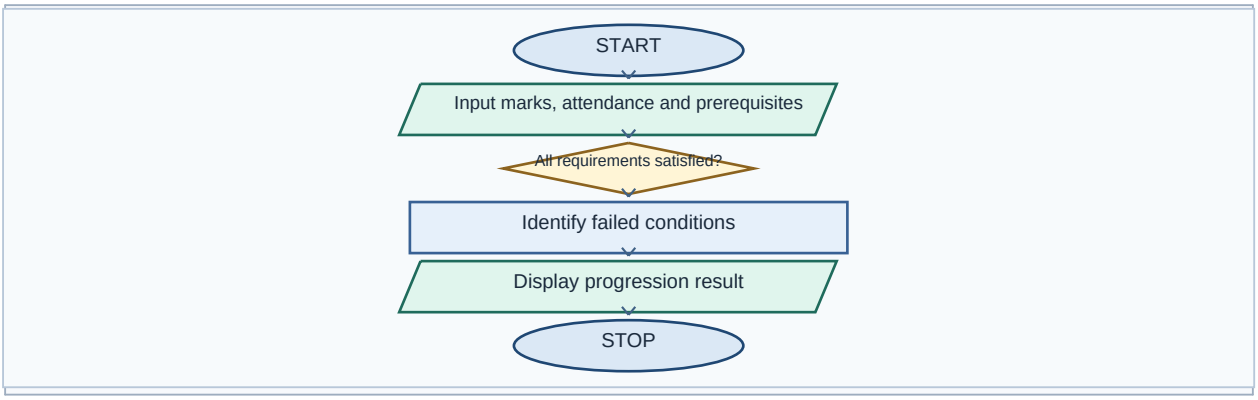
6. If attendance is below 75, display attendance shortage.

7. If prerequisites are not completed, display prerequisites incomplete.

8. If all conditions are satisfied, display eligible for progression.

9. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT marks
INPUT attendance
INPUT prereq
IF marks < 40
```

```
    DISPLAY "Subject not passed"
END IF
IF attendance < 75
    DISPLAY "Attendance shortage"
END IF
IF prereq != "yes"
    DISPLAY "Prerequisites incomplete"
END IF
IF marks >= 40 AND attendance >= 75 AND prereq = "yes"
    DISPLAY "Eligible for progression"
END IF
STOP
```

7. Python Source Code

```
# Academic progression
marks = int(input("Enter semester marks: "))
attendance = int(input("Enter attendance %: "))
prereq = input("Prerequisites completed (yes/no): ")

if marks < 40:
    print("Subject not passed")
if attendance < 75:
    print("Attendance shortage")
if prereq != "yes":
    print("Prerequisites incomplete")

if marks >= 40 and attendance >= 75 and prereq == "yes":
    print("Eligible for progression")
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	Marks 70, attendance 80, prerequisites yes	All conditions satisfied | — | — | Eligible for progression
2	Marks 35, attendance 80, prerequisites yes	Marks below 40 | — | — | Subject not passed
3	Marks 65, attendance 70, prerequisites no	Attendance and prerequisites fail | — | — | Both reasons shown

9. Output

```
Enter semester marks: 72
Enter attendance %: 85
Prerequisites completed (yes/no): yes
Eligible for progression
```

10. Conclusion

The program evaluates academic progression using marks, attendance, and prerequisite completion. It demonstrates multiple independent conditions and combined logical expressions.

8. Examination Seating Arrangement

1. Problem Statement

Write a Python program to generate a seating layout containing seven rows and four columns. Mark one specific seat (row 2, column 2) as Reserved. For all other positions, display a seat label when the sum of the zero-based row and column indices is even; otherwise display --.

2. Objectives

• Create a seating grid with seven rows and four columns.

• Use nested loops to visit every seat position.

• Mark the specified seat as Reserved.

• Apply an alternating pattern to the remaining seats.

3. Assumptions

• The seating arrangement has exactly seven rows and four columns.

• Python indexing begins at zero.

• The reserved position is (1, 1), corresponding to row 2 and column 2 for people.

• A seat label is displayed when (row index + column index) is even.

4. Algorithm

1. Start.

2. Set rows to 7 and columns to 4.

3. For each row, iterate through every column.

4. If the position is (1, 1), print Reserved.

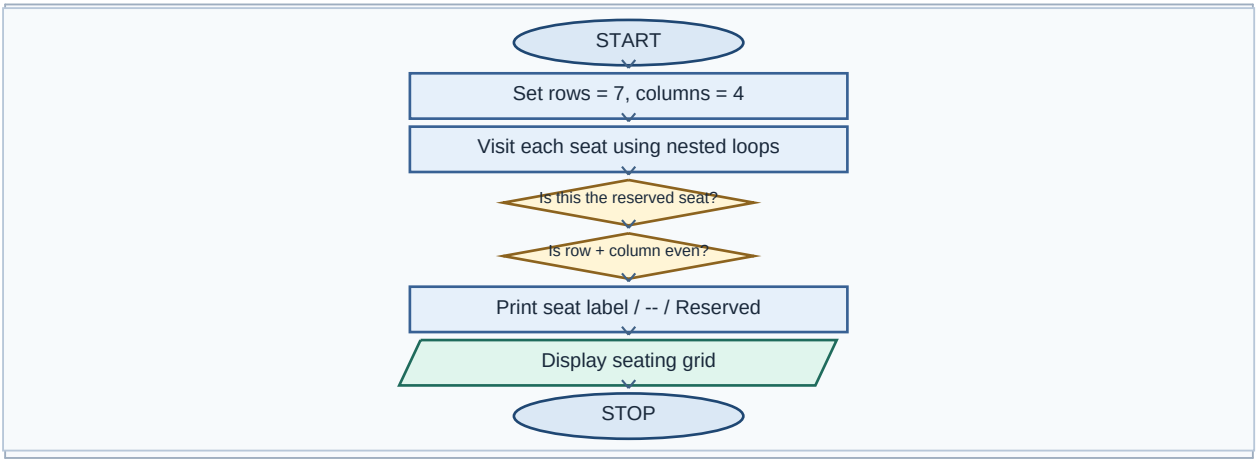
5. Otherwise, if the sum of row and column indices is even, print the seat label.

6. Otherwise print --.

7. After each row, move to a new line.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
rows ← 7
cols ← 4
FOR i from 0 to rows - 1
```

```
    FOR j from 0 to cols - 1
        IF i = 1 AND j = 1
            PRINT "Reserved" without newline
        ELSE IF (i + j) MOD 2 = 0
            PRINT seat label R(i+1)C(j+1) without newline
        ELSE
            PRINT "--" without newline
        END IF
    END FOR
    PRINT newline
END FOR
STOP
```

7. Python Source Code

```
# Examination seating
rows = 7
cols = 4

for i in range(rows):
    for j in range(cols):
        if (i, j) == (1, 1):
            print("Reserved", end="\t")
        elif (i + j) % 2 == 0:
            print(f"R{i+1}C{j+1}", end="\t")
        else:
            print("--", end="\t")
    print()
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	Standard 7 × 4 layout	7 rows | 4 columns | (2,2) reserved | Pattern printed
2	Check row 2, column 2	— | — | Reserved | Reserved
3	Check row 1, column 1	— | — | R1C1 | R1C1

9. Output

```
R1C1    --      R1C3    --
--      Reserved --      R2C4
R3C1    --      R3C3    --
--      R4C2    --      R4C4
R5C1    --      R5C3    --
--      R6C2    --      R6C4
R7C1    --      R7C3    --
```

10. Conclusion

The program creates a patterned examination seating arrangement and reserves a selected seat. It demonstrates nested for loops, conditional statements, modulo operation, and formatted output.

9. Food Supply Chain Stock Reconciliation

1. Problem Statement

Develop a Python program to reconcile food stock at three locations: College, Hostel, and Canteen. For each location, accept the quantity received, distributed, and damaged. Calculate the remaining stock, request replenishment when stock is below 10, and report a stock discrepancy when the remaining stock is negative.

2. Objectives

• Process stock data for three locations.

• Calculate remaining stock after distribution and damage.

• Identify locations that need replenishment.

• Detect negative stock balances as discrepancies.

3. Assumptions

• The locations are College, Hostel, and Canteen.

• Received, distributed, and damaged quantities are entered as integers.

• Remaining stock is received minus distributed minus damaged.

• The replenishment threshold is 10 units.

• The supplied code checks replenishment before checking negative stock, so a negative balance can trigger both messages.

4. Algorithm

1. Start.

2. Create a list of the three locations.

3. For each location, read received, distributed, and damaged quantities.

4. Calculate remaining = received – distributed – damaged.

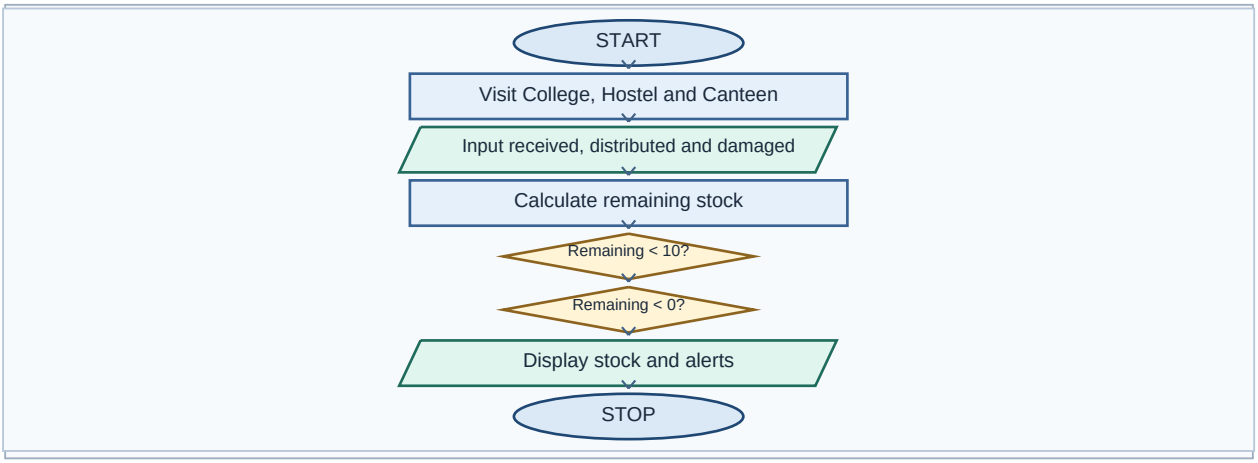
5. Display the location and remaining quantity.

6. If remaining is less than 10, display the replenishment quantity as 10 – remaining.

7. If remaining is negative, display stock discrepancy detected.

8. Repeat for all locations and stop.

5. Flowchart



6. Pseudocode

```
START
```

```
locations ← ["College", "Hostel", "Canteen"]
FOR each place in locations
    INPUT received
    INPUT distributed
    INPUT damaged
    remaining ← received - distributed - damaged
    DISPLAY place, remaining
    IF remaining < 10
        DISPLAY "Replenishment needed", 10 - remaining
    END IF
    IF remaining < 0
        DISPLAY "Stock discrepancy detected"
    END IF
END FOR
STOP
```

7. Python Source Code

```
# Food Supply Chain Stock Reconciliation
locations = ["College", "Hostel", "Canteen"]

for place in locations:
    received = int(input("Quantity received: "))
    distributed = int(input("Quantity distributed: "))
    damaged = int(input("Quantity damaged: "))
    remaining = received - distributed - damaged

    print(place, "Remaining:", remaining)
    if remaining < 10:
        print("Replenishment needed:", 10 - remaining)
    if remaining < 0:
        print("Stock discrepancy detected")
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	Received 100, distributed 70, damaged 5	25 | No | No | 25 units
2	Received 50, distributed 45, damaged 2	3 | Yes: 7 units | No | 3 units
3	Received 20, distributed 25, damaged 0	-5 | Yes: 15 units | Yes | Discrepancy detected

9. Output

```
College
Quantity received: 100
Quantity distributed: 70
Quantity damaged: 5
College Remaining: 25
Hostel
Quantity received: 50
Quantity distributed: 45
Quantity damaged: 2
Hostel Remaining: 3
Replenishment needed: 7
Canteen
Quantity received: 20
Quantity distributed: 25
Quantity damaged: 0
Canteen Remaining: -5
Replenishment needed: 15
Stock discrepancy detected
```

10. Conclusion

The program calculates remaining stock for each location and identifies low stock and negative balances. It demonstrates lists, loops, arithmetic, and conditional checks.

10. Calendar Display

1. Problem Statement

Create a Python program that prints a simple monthly calendar layout for a given number of days and starting weekday. The user enters the number of days and the weekday index, where Monday is 0 and Sunday is 6. The program aligns the first date under the correct weekday and prints all dates in sequence.

2. Objectives

• Accept the number of days in the month.

• Accept the starting weekday index.

• Print weekday headings.

• Indent the first week according to the starting weekday.

• Display all dates in a calendar-like arrangement.

3. Assumptions

• The number of days is a positive integer.

• The starting weekday is between 0 and 6.

• Monday is represented by 0 and Sunday by 6.

• The program prints a simple calendar and does not validate real month lengths.

4. Algorithm

1. Start.

2. Read the number of days.

3. Read the starting weekday from 0 to 6.

4. Print weekday headings from Monday to Sunday.

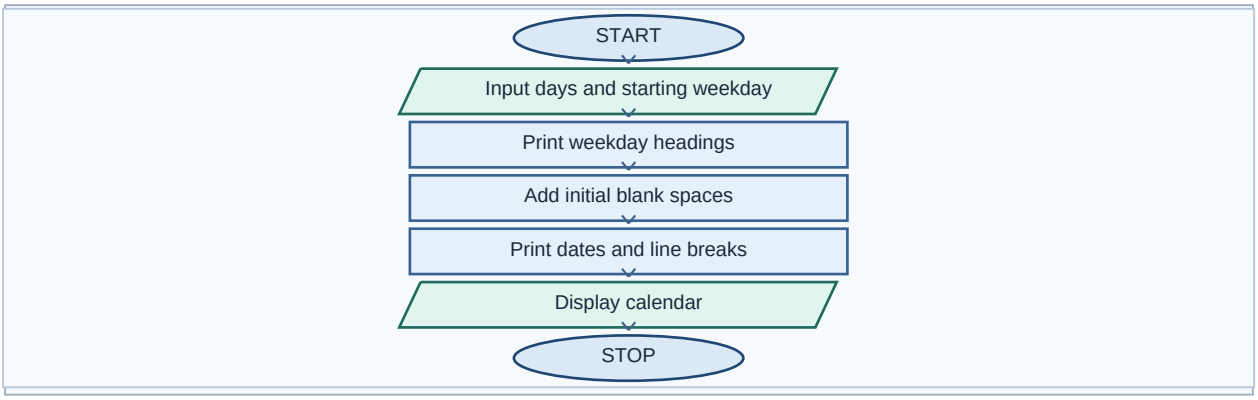
5. Print blank spaces for the weekdays before day 1.

6. For each day from 1 to the entered number of days, print the day number.

7. After completing a week, move to a new line.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT days
INPUT start
PRINT "Mon Tue Wed Thu Fri Sat Sun"
FOR i from 1 to start
```

```
    PRINT blank calendar space
END FOR
FOR day from 1 to days
    PRINT day in calendar format
    IF (start + day) MOD 7 = 0
        PRINT newline
    END IF
END FOR
STOP
```

7. Python Source Code

```
days = int(input("Enter number of days: "))
start = int(input("Starting weekday (0=Mon, 6=Sun): "))

print("Mon Tue Wed Thu Fri Sat Sun")
for i in range(start):
    print("    ", end="")

for day in range(1, days + 1):
    print(f"{day:3}", end=" ")
    if (start + day) % 7 == 0:
        print()
```

8. Test Cases

Test	Input / Condition	Expected / Actual Result
1	30 days; start weekday 0 (Monday)	30 | 0 | Starts under Mon | Calendar aligned
2	31 days; start weekday 5 (Saturday)	31 | 5 | Five blank spaces first | Calendar aligned
3	28 days; start weekday 6 (Sunday)	28 | 6 | Six blank spaces first | Calendar aligned

9. Output

```
Enter number of days: 10
Starting weekday (0=Mon, 6=Sun): 2
Mon Tue Wed Thu Fri Sat Sun
            1   2   3   4   5
  6   7   8   9  10
```

10. Conclusion

The program displays dates in a simple weekly calendar format based on the supplied starting weekday. It demonstrates loops, range(), formatted strings, and modulo-based week transitions.